CAD ELT walkthrough

JPL Close Approach Data pipeline: API to Snowflake Bronze, Dynamic Tables (Silver/Gold), Streamlit on Gold, scheduled with GitHub Actions. ELT: Python extracts and loads; transform stays in Snowflake.

Repo: src/ (Python), sql/ (Snowflake), .github/workflows/cad-elt.yml (cron).

What CAD is

NASA/JPL Close Approach Data lists predicted times when near-Earth asteroids (and some comets) pass Earth: designation, time, miss distance, relative speed. It is an ephemeris (calculated flybys), not a live radar feed. A weekly extract is enough.

GET https://ssd-api.jpl.nasa.gov/cad.api with a date window, dist-max=0.05 AU, nea=true. Bronze keeps one JSON document: extracted_at + payload VARIANT.

End-to-end flow

```
JPL CAD API
    |  extract_cad_elt.py          (GitHub runner, Mondays 06:00 UTC)
    v
JSON on the runner (not committed)
    |  load_cad_snowflake.py       warehouse: SNOWFLAKE_LEARNING_WH
    v
ELT_DEV.BRONZE.CAD_RAW             one row, full JSON
    |  refresh_cad_dts.py          ALTER DYNAMIC TABLE ... REFRESH
    v
SILVER.CAD  ->  GOLD.CAD_FLYBYS / CAD_KPI / CAD_BY_DAY
    |  src/app.py                  warehouse: DASHBOARD_WH
    v
Streamlit (local): KPIs, flyby table, approaches per day
```

1. Extract

src/extract_cad_elt.py: requests.get with a User-Agent, query params, JSON dump to data/raw/cad/cad_<UTC stamp>.json. The raw folder is gitignored. On GitHub Actions the file exists only for the job, then the runner is gone.

2. Load (Bronze)

src/load_cad_snowflake.py uses SNOWFLAKE_WAREHOUSE (load WH, not the dashboard WH). CREATE TABLE IF NOT EXISTS CAD_RAW (extracted_at STRING, payload VARIANT). DELETE then INSERT ... PARSE_JSON so Bronze stays one row (latest snapshot). Proof of a run is extracted_at, e.g. cad_20261019T120645Z, not a growing row count.

3. Transform (Dynamic Tables)

DDL: sql/cad_dynamic_tables.sql. sql/cad_archive.sql is the learning path (CTAS, then DROP, then DTs) and must not be run as a whole.

SILVER.CAD: flatten payload:data, TARGET_LAG = 3 days. GOLD.CAD_FLYBYS / CAD_KPI / CAD_BY_DAY: TARGET_LAG = DOWNSTREAM. All DTs use SNOWFLAKE_LEARNING_WH.

Why 3-day lag if Actions already run Mondays? Transform lives in Snowflake. Lag is a safety net if the Action fails. ALTER DYNAMIC TABLE ... REFRESH in the Action makes Gold match Bronze on Monday. src/refresh_cad_dts.py runs Silver first, then Gold.

4. Orchestration (GitHub Actions)

.github/workflows/cad-elt.yml: cron 0 6 * * 1 (Monday 06:00 UTC) plus workflow_dispatch. Steps: checkout, Python 3.12, pip install, extract, load, refresh. Secrets match .env keys except the dashboard warehouse. After a YAML change use Run workflow, not Re-run of an old job.

5. Two warehouses

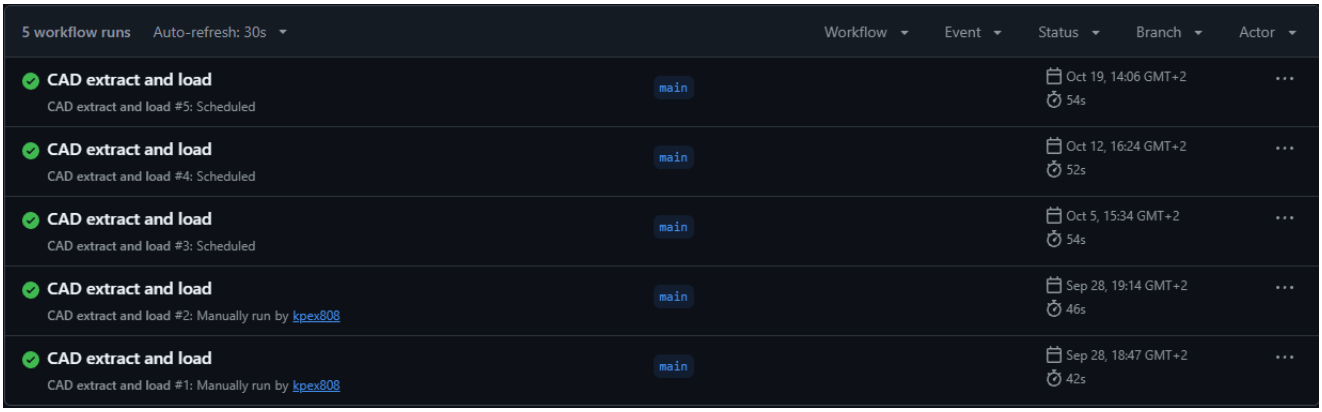
SNOWFLAKE_WAREHOUSE (SNOWFLAKE_LEARNING_WH): load and DT refresh. SNOWFLAKE_DASHBOARD_WAREHOUSE (DASHBOARD_WH): Streamlit SELECTs only. DASHBOARD_WH is XSMALL, AUTO_SUSPEND 60, AUTO_RESUME true so a dashboard rerun does not wake the load warehouse. Credits only while running.

6. Dashboard

src/app.py via: python -m streamlit run src/app.py. Reads GOLD.CAD_KPI, CAD_FLYBYS (closest 100), CAD_BY_DAY. Streamlit is not scheduled. It only reads Gold.

Screenshots

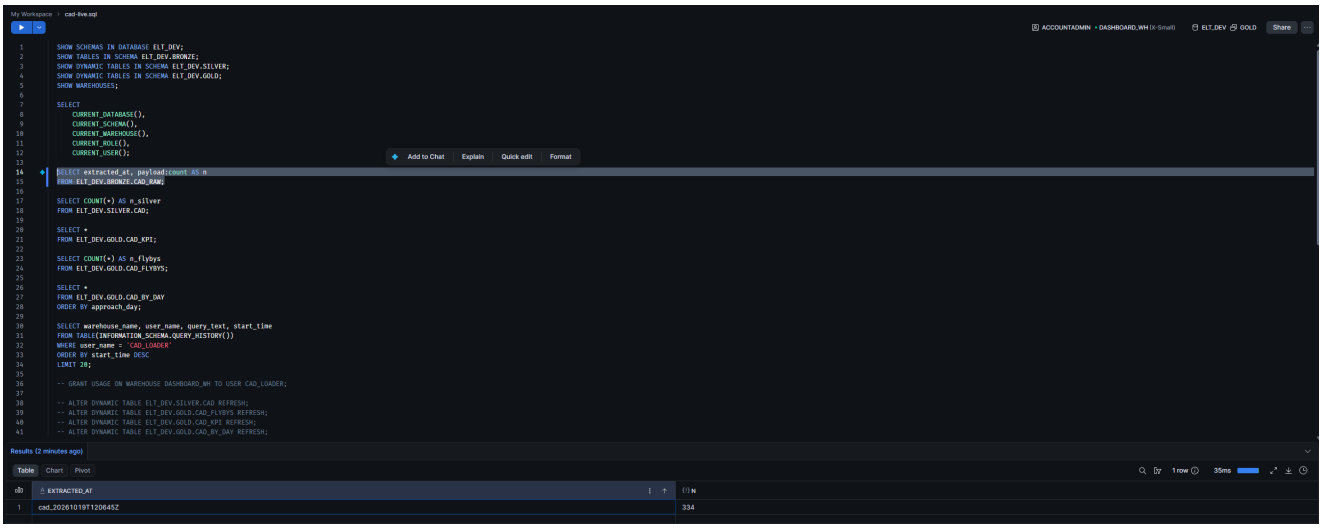
GitHub Actions — weekly runs



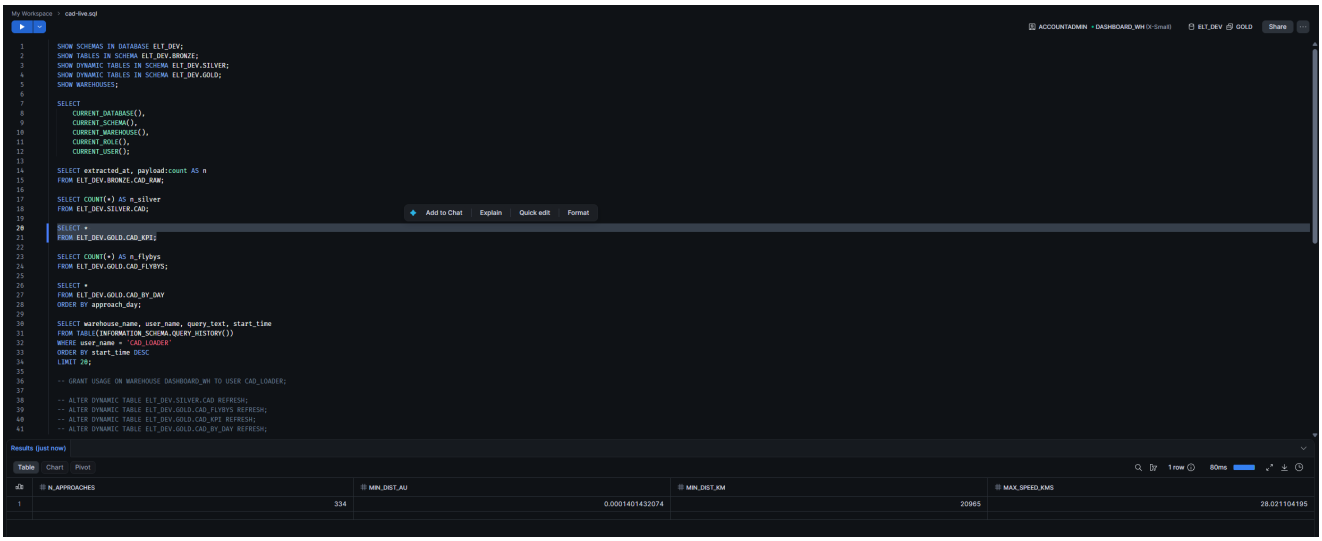
GitHub Actions — job steps



Snowflake Bronze — extracted_at



Snowflake Gold KPI



```
SHOW SCHEMAS IN DATABASE ELT_DEV;
SHOW TABLES IN SCHEMA ELT_DEV.BRONZE;
SHOW DYNAMIC TABLES IN SCHEMA ELT_DEV.SILVER;
SHOW DYNAMIC TABLES IN SCHEMA ELT_DEV.GOLD;
SHOW WAREHOUSES;

SELECT
    CURRENT_DATABASE(),
    CURRENT_SCHEMA(),
    CURRENT_WAREHOUSE(),
    CURRENT_ROLE(),
    CURRENT_USER();

SELECT extracted_at, payload:count AS n
FROM ELT_DEV.BRONZE.CAD_RAW;

SELECT COUNT(*) AS n_silver
FROM ELT_DEV.SILVER.CAD;

SELECT *
FROM ELT_DEV.GOLD.CAD_KPI;

SELECT COUNT(*) AS n_flybys
FROM ELT_DEV.GOLD.CAD_FLYBYS;

SELECT *
FROM ELT_DEV.GOLD.CAD_BY_DAY
ORDER BY approach_day;

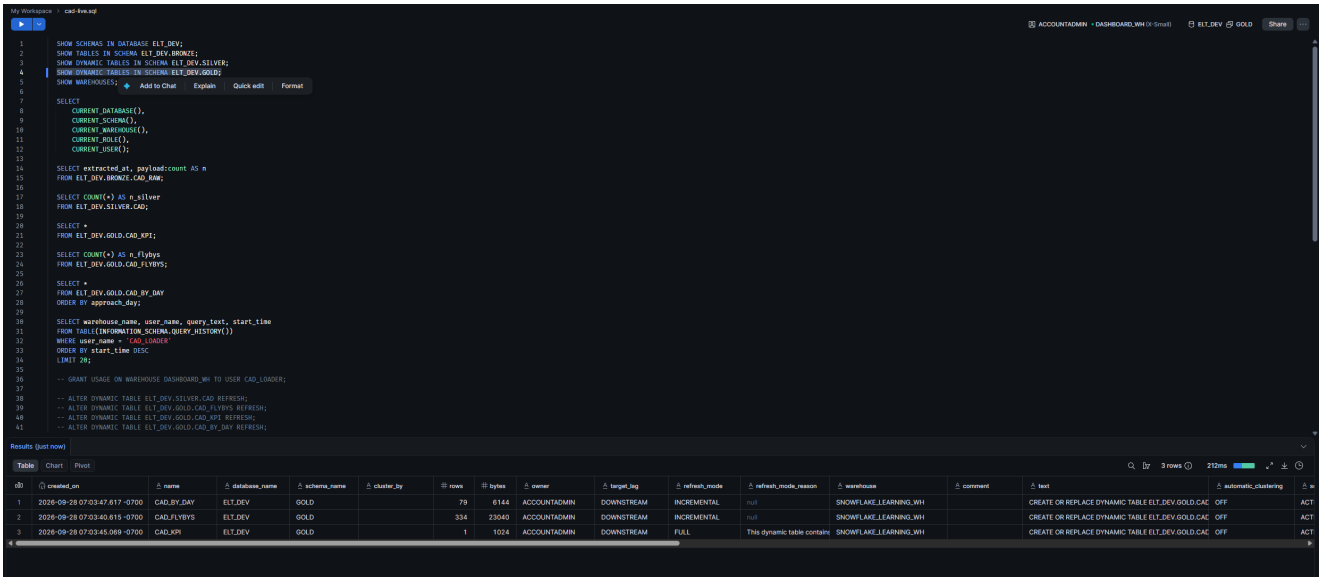
SELECT warehouse_name, user_name, query_text, start_time
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE user_name = 'CAD_LOADER'
ORDER BY start_time DESC
LIMIT 20;

-- GRANT USAGE ON WAREHOUSE DASHBOARD_WH TO USER CAD_LOADER;

-- ALTER DYNAMIC TABLE ELT_DEV.SILVER.CAD REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_FLYBYS REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_KPI REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_BY_DAY REFRESH;
```

N_APPROACHES	MIN_DIST_AU	MIN_DIST_KM	MAX_SPEED_KMS
334	0.0001401432074	20965	28.021104195

Snowflake Dynamic Tables (Gold)



```
SHOW SCHEMAS IN DATABASE ELT_DEV;
SHOW TABLES IN SCHEMA ELT_DEV.BRONZE;
SHOW DYNAMIC TABLES IN SCHEMA ELT_DEV.SILVER;
SHOW DYNAMIC TABLES IN SCHEMA ELT_DEV.GOLD;
SHOW WAREHOUSES;

SELECT
    CURRENT_DATABASE(),
    CURRENT_SCHEMA(),
    CURRENT_WAREHOUSE(),
    CURRENT_ROLE(),
    CURRENT_USER();

SELECT extracted_at, payload:count AS n
FROM ELT_DEV.BRONZE.CAD_RAW;

SELECT COUNT(*) AS n_silver
FROM ELT_DEV.SILVER.CAD;

SELECT *
FROM ELT_DEV.GOLD.CAD_KPI;

SELECT COUNT(*) AS n_flybys
FROM ELT_DEV.GOLD.CAD_FLYBYS;

SELECT *
FROM ELT_DEV.GOLD.CAD_BY_DAY
ORDER BY approach_day;

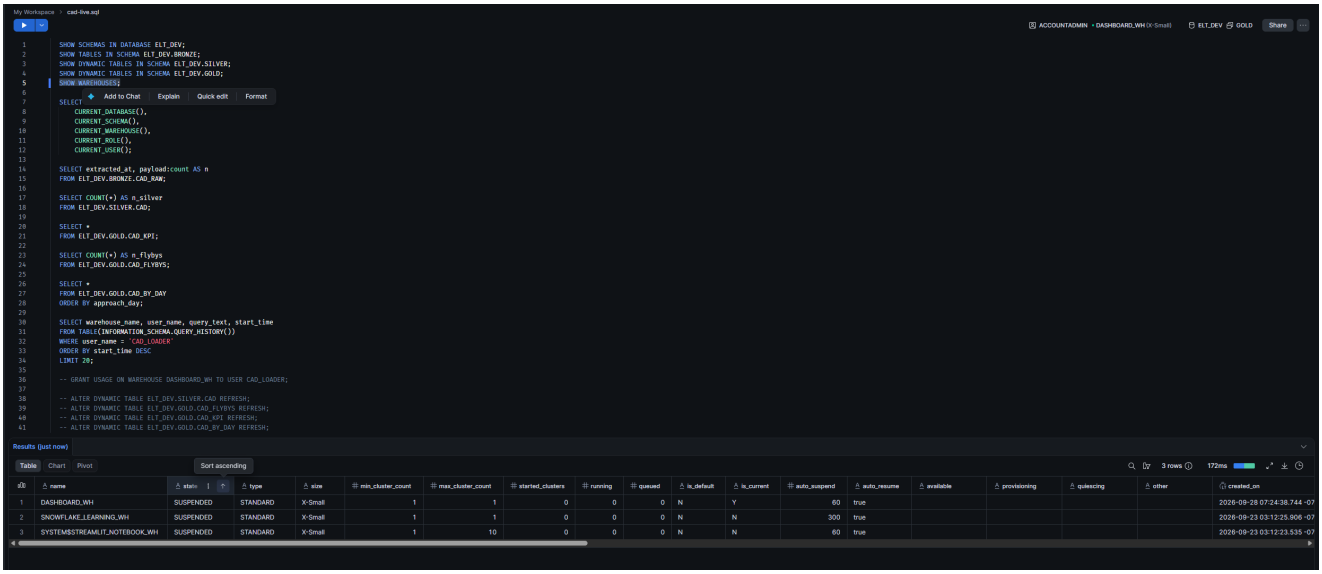
SELECT warehouse_name, user_name, query_text, start_time
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE user_name = 'CAD_LOADER'
ORDER BY start_time DESC
LIMIT 20;

-- GRANT USAGE ON WAREHOUSE DASHBOARD_WH TO USER CAD_LOADER;

-- ALTER DYNAMIC TABLE ELT_DEV.SILVER.CAD REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_FLYBYS REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_KPI REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_BY_DAY REFRESH;
```

	created_on	name	database_name	schema_name	cluster_by	rows	bytes	owner	target_lag	refresh_mode	refresh_mode_reason	warehouse	comment	text	automatic_clustering
1	2026-09-28 07:03:47.617 -0700	CAD_BY_DAY	ELT_DEV	GOLD		79	6144	ACCOUNTADMIN	DOWNSTREAM	INCREMENTAL	null	SNOWFLAKE_LEARNING_WH		CREATE OR REPLACE DYNAMIC TABLE ELT_DEV.GOLD.CAD	OFF
2	2026-09-28 07:03:40.615 -0700	CAD_FLYBYS	ELT_DEV	GOLD		334	23040	ACCOUNTADMIN	DOWNSTREAM	INCREMENTAL	null	SNOWFLAKE_LEARNING_WH		CREATE OR REPLACE DYNAMIC TABLE ELT_DEV.GOLD.CAD	OFF
3	2026-09-28 07:03:45.069 -0700	CAD_KPI	ELT_DEV	GOLD		1	1024	ACCOUNTADMIN	DOWNSTREAM	FULL	This dynamic table contains	SNOWFLAKE_LEARNING_WH		CREATE OR REPLACE DYNAMIC TABLE ELT_DEV.GOLD.CAD	OFF

Warehouses



```
SHOW SCHEMAS IN DATABASE ELT_DEV;
SHOW TABLES IN SCHEMA ELT_DEV.BRONZE;
SHOW DYNAMIC TABLES IN SCHEMA ELT_DEV.SILVER;
SHOW DYNAMIC TABLES IN SCHEMA ELT_DEV.GOLD;
SHOW WAREHOUSES;

SELECT
    CURRENT_DATABASE(),
    CURRENT_SCHEMA(),
    CURRENT_WAREHOUSE(),
    CURRENT_ROLE(),
    CURRENT_USER();

SELECT extracted_at, payload:count AS n
FROM ELT_DEV.BRONZE.CAD_RAW;

SELECT COUNT(*) AS n_silver
FROM ELT_DEV.SILVER.CAD;

SELECT *
FROM ELT_DEV.GOLD.CAD_KPI;

SELECT COUNT(*) AS n_flybys
FROM ELT_DEV.GOLD.CAD_FLYBYS;

SELECT *
FROM ELT_DEV.GOLD.CAD_BY_DAY
ORDER BY approach_day;

SELECT warehouse_name, user_name, query_text, start_time
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE user_name = 'CAD_LOADER'
ORDER BY start_time DESC
LIMIT 20;

-- GRANT USAGE ON WAREHOUSE DASHBOARD_WH TO USER CAD_LOADER;

-- ALTER DYNAMIC TABLE ELT_DEV.SILVER.CAD REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_FLYBYS REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_KPI REFRESH;
-- ALTER DYNAMIC TABLE ELT_DEV.GOLD.CAD_BY_DAY REFRESH;
```

Results (just now)

	name	state	type	size	min_cluster_count	max_cluster_count	started_clusters	running	queued	is_default	is_current	auto_suspend	auto_resume	available	provisioning	quiescing	other	created_on
1	DASHBOARD_WH	SUSPENDED	STANDARD	X-Small	1	1	0	0	0	N	Y	60	true					2026-09-28 07:24:38.744 -07
2	SNOWFLAKE_LEARNING_WH	SUSPENDED	STANDARD	X-Small	1	1	0	0	0	N	N	300	true					2026-09-23 03:12:25.906 -07
3	SYSTEM$STREAMLIT_NOTEBOOK_WH	SUSPENDED	STANDARD	X-Small	1	10	0	0	0	N	N	60	true					2026-09-23 03:12:23.535 -07

Streamlit dashboard

